

AI APPENDIX & REFLECTION

Car Park Management Simulation - Operating Systems Project

Institution: Royal University of Phnom Penh

Department: IT Engineering

Course: Operating Systems

Instructor: Kor Sokchea, Master

1. Tools Used & Timing

Tools Used: Gemini and ChatGPT.

When & How They Were Used: Utilized during the design and coding phase to structure the Model-View-Service-Thread architecture, establish Java Swing layout components, and implement thread synchronization logic using `ReentrantLock` and `Condition` variables.

2. Prompts, Accepted & Rejected Snippets

- **Prompt 1:** *"Implement a thread-safe bounded buffer in Java using `ReentrantLock` and `Condition` variables for a car park simulation."*
 - **Accepted Snippet:** The fair lock implementation with `notFull` and `notEmpty` condition variables to manage producer blocking and consumer signaling without CPU spinlocks.
- **Prompt 2:** *"Design a Java Swing UI layout that displays an arena grid for parking slots, an LED traffic sign, telemetry stats, and a terminal log."*
 - **Accepted Snippet:** Modular UI separation (`ArenaPanel`, `LogPanel`, `ControlPanel`, `Dashboard`) paired with `SwingUtilities.invokeLater()` to safely update components from background threads.
- **Rejected Suggestion:** Initial AI-suggested naive busy-waiting loops with `while` statements and no condition variable `await()` calls.
 - **Reason for Rejection:** Rejected because it causes 100% CPU core utilization overhead and fails operating system concurrency design requirements.

3. Brief Reflection

What AI Got Wrong: Initial concurrency suggestions occasionally lacked proper thread interruption handling or risked missed signals if condition checks were executed with `if` statements instead of proper `while` loops.

How the Student Verified/Modified: Verified the implementation by testing boundary states (e.g., setting buffer capacity to 4 with a high production rate). Modified the code to incorporate `while` loops around `await()` calls to handle spurious wakeups and ensure thread safety.

What Was Learned: Gained a deep understanding of how monitor conditions prevent race conditions between incoming vehicle check-ins and outgoing vehicle check-outs, and how crucial proper thread blocking and unparking mechanisms are in operating systems.